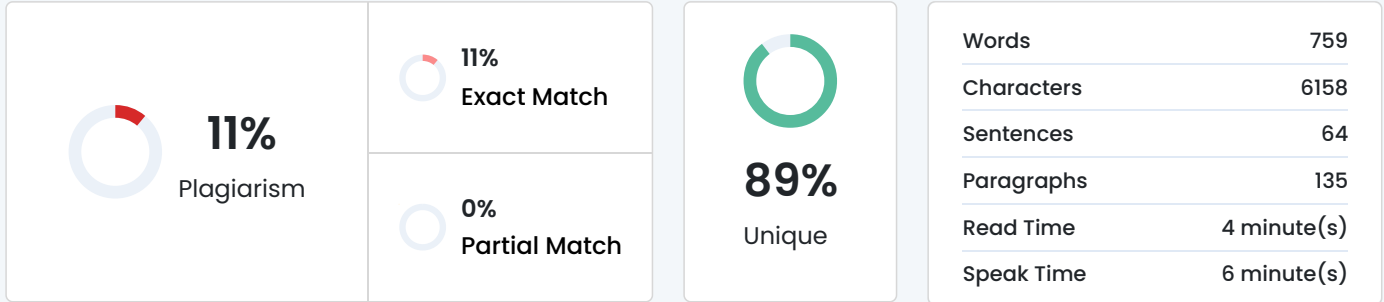


Plagiarism Scan Report



Content Checked For Plagiarism

ASSIGNMENT REPORT

EMPLOYEE PAYROLL AND ATTENDANCE MANAGEMENT
SYSTEM

Course: CSA08 / Programming in Python

Department: Computer Science / Information Technology

Program: BE

Maximum Marks: 100

Submitted By:

Rohith.s(1972260062)

bandi tejaswini (1973260009) _____

Faculty Name: __

bRohith Bhat _____

1. Problem Statement

Develop a Python-based Employee Payroll and Attendance Management System to manage employee records,

track daily attendance, handle leave and overtime, calculate monthly salaries, search employee information, generate analytical reports, and store data using CSV files.

2. Objective

- To automate employee attendance and payroll management.
- To reduce manual calculation errors in salaries and deductions.
- To use Python data structures for efficient storage and retrieval.
- To generate useful attendance and payroll reports.
- To demonstrate Python fundamentals, control flow, functions, strings, file handling and exception handling.

3. Requirements and Environment Used

Hardware: Computer/Laptop with minimum 4 GB RAM.

Software: Python 3.x and VS Code/IDLE/PyCharm.

Libraries: csv, datetime, statistics.

Data Structures: Dictionaries for employees and attendance, lists for history, tuples for fixed information, and sets for unique departments/designations.

4. Design / Proposed Solution

The application uses Employee ID as a unique key. Attendance is recorded by date and status. Salary is calculated using basic pay, deductions, overtime pay and bonus. Reports analyze attendance and department-wise payroll cost.

System Flow: Main Menu * Employee Management * Attendance * Leave & Overtime * Salary Calculation
* Reports * CSV File Operations

5. Algorithm / Pseudocode

FUNCTION calculate_salary(emp_id):

```

IF emp_id not in employees: raise InvalidEmployeeError
basic_pay = employee basic pay
absent_days = count Absent attendance
deduction = absent_days * (basic_pay / 30)
overtime_pay = overtime_hours * overtime_rate
bonus = basic_pay * bonus_rate
RETURN basic_pay - deduction + overtime_pay + bonus
FUNCTION mark_attendance(emp_id, date, status):
    IF status is invalid: raise InvalidAttendanceStatusError
    store attendance record
    RETURN success

```

6. Implementation / Source Code

```

import csv
class InvalidAttendanceStatusError(Exception):
    pass
employees = {}
attendance = {}
overtime = {}
def add_employee(emp_id, name, department, designation, basic_pay):
    employees[emp_id] = {
        "name": name.title(),
        "department": department.title(),
        "designation": designation.title(),
        "basic_pay": float(basic_pay)
    }
def mark_attendance(emp_id, date, status):
    valid = {"Present", "Absent", "Leave", "Half-day"}
    if status not in valid:
        raise InvalidAttendanceStatusError("Invalid attendance status")
    if emp_id not in employees:
        raise ValueError("Employee not found")
    attendance.setdefault(emp_id, {})[date] = status
def add_overtime(emp_id, hours):
    overtime[emp_id] = overtime.get(emp_id, 0) + float(hours)
def calculate_salary(emp_id):
    if emp_id not in employees:
        raise ValueError("Employee not found")
    basic = employees[emp_id]["basic_pay"]
    records = attendance.get(emp_id, {})
    absent = sum(1 for s in records.values() if s == "Absent")
    deduction = absent * (basic / 30)
    overtime_pay = overtime.get(emp_id, 0) * 100
    bonus = basic * 0.05 if absent == 0 else 0
    return basic - deduction + overtime_pay + bonus
def search_by_department(department):
    return {k:v for k,v in employees.items()
            if v["department"].lower() == department.lower()}
def save_employees():
    with open("employees.csv", "w", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(["ID", "Name", "Department",
                        "Designation", "Basic Pay"])
    for emp_id, data in employees.items():
        writer.writerow([emp_id, data["name"],

```

```

        data["department"],
        data["designation"],
        data["basic_pay"]])
add_employee("E001", "Ravi Kumar", "IT", "Developer", 30000)
add_employee("E002", "Anitha", "HR", "Executive", 28000)
mark_attendance("E001", "2026-08-01", "Present")
mark_attendance("E002", "2026-08-01", "Absent")
add_overtime("E001", 10)
print("E001 Salary:", calculate_salary("E001"))
print("IT Employees:", search_by_department("IT"))
save_employees()

```

7. Test Cases and Results

Test Case Scenario Expected Result

TC1 Add valid employee Employee added successfully

TC2 Mark Present attendance Attendance recorded

TC3 Calculate salary without absence Full salary with applicable bonus

TC4 Employee has absence Deduction applied correctly

TC5 Search by department Matching employees displayed

TC6 Invalid attendance status Custom exception handled

TC7 Missing/invalid employee Error handled gracefully

8. Execution Screenshots / Output

Attach screenshots showing employee addition, attendance marking, salary calculation, report generation, CSV

output and exception handling.

9. Analysis and Discussion

Dictionaries provide fast lookup using Employee ID. Nested dictionaries make date-wise attendance retrieval efficient. Lists are suitable for chronological histories, while sets maintain unique department and designation values. Modular functions improve readability, testing and maintenance. CSV files provide simple persistence for a prototype.

10. Conclusion

This project develops a Python-based Employee Payroll and Attendance Management System. It applies Python

fundamentals, functions, control flow, string operations, dictionaries, lists, tuples, sets, file handling and exception

handling. The system manages employee information, records attendance, calculates salaries and produces reports. Future improvements can include a GUI, database integration, authentication and advanced analytics.

11. Individual Contribution of Group Members

Member Contribution

Member 1 Employee record management and system design

Member 2 Attendance and leave management

Member 3 Payroll and overtime calculation

Member 4 Testing, reports, documentation and screenshots

12. References

- Python Documentation. Python 3.x Library Reference.
- Downey, A. B. **Think Python: How to Think Like a Computer Scientist.**
- Severance, C. Python for Everybody.
- Course assignment guidelines and the provided Employee Payroll and Attendance Management System assignment document.

Similarity 3%

Title:[Get_worker in client.run: raise ValueError\("No worker found ...](#)

Feb 11, 2025 — run: raise ValueError("No worker found") from None · Distributed ... Client does not return workers, Job dies quickly · Distributed.[Read more](#)

[Check](#)

Similarity 2%

Title:[SAAMA Professional Designation Application](#)

Designation title Description of designation - Designation abbreviation Accrediting body Registration number Year awarded

https://google.com/goto?url=CAESWAHrOzAVOuQLDxa8c4Nzl_G1Bu8zcUO49-96MdZ04Pw7DO8HRG-F5-1Yjipw sfci3-P-cYru_GGzPPnumnpLHmA86FKe7S2q6UczPs8tELrsXWtoWGF49ml

Similarity 2%

Title:[Writing CSV files in Python](#)

Jul 12, 2025 — ... =") as file: writer = csv.writer(file, quoting=csv.QUOTE_NONNUMERIC, delimiter='|', quotechar='~') writer.writerows(rows) print(f"Data has ...[Read more](#)

https://google.com/goto?url=CAESeQHrOzAV1JKfKBSf-FwVSRr_lI4JwKy52lyQwbfBbkfrVdOy0nH4MUmlQY8O4fd lg3WwNOFFXtlvpzT6DFN6_e8h5kLpVbg9RaIpc7AvbU9aIHlusYR3b6Du_ZFPKP4Okhvva7ChwSDSOW7FLotCA94-m_6YUIGrchY

Similarity 2%

Title:[2026 Military Pay Chart: All Grades and Years of Service](#)

May 11, 2026 — Pay grade is your rank designation (E-1 through E-9 for enlisted, W-1 through W-5 for warrant officers, O-1 through O-10 for commissioned ...[Read more](#)

https://google.com/goto?url=CAESZAHrOzAVIa2ntS0_1eB2rtiRJJjq0TDBazxGDFIlq6bg5_bh6dIDs2kKjAfmcvHKKx lnpn8nyUVd-Kv_t08oRx8gFCzUaXNIIP5VdBeIu-bl2QOHhlojIwF0mbf04eFJuakr8SQ

Similarity 2%

Title:[Associate in Insurance Data Analytics® \(AIDA™\)](#)

Gain in-demand data analytics skills and learn how to speak the language of data with the Associate in Insurance Data Analytics® (AIDA™) designation.

[Check](#)

Similarity 2%

Title:[Think Python: How to Think Like a Computer Scientist](#)

Think Python is an introduction to Python programming for beginners. It starts with basic concepts of programming, and is carefully designed to define all terms ...[Read more](#)

<https://google.com/goto?url=CAEScgHrOzAVivImUEtUgm0Ko4Px8pF48POb7uPSvyy5EYqVaFPpeVrvkGQlqJE3Vl 3uTtzv2dEclu-fm1-A2DaMeMsHddcwj835NuWltzCD7oM-SZKMT3tMleov4CizzKFGUoAzVlflGxHTsM2IaOutvKHMSg>
